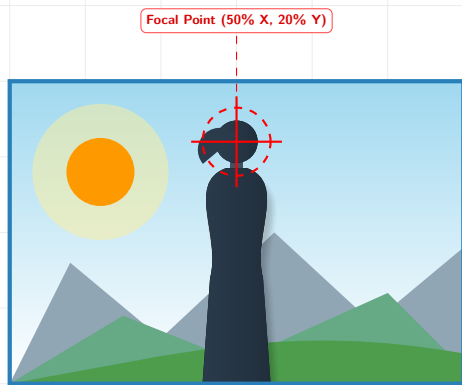


The Mechanics of object-fit: cover

How the rendering engine scales, crops, and anchors focal points (object-position) in responsive CSS bounding boxes.

1. Source Image

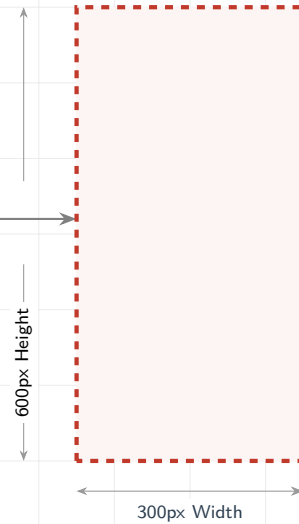
Intrinsic Size: 600px × 400px (3:2)



2. Target CSS Box

Box Size: 300px × 600px (1:2)

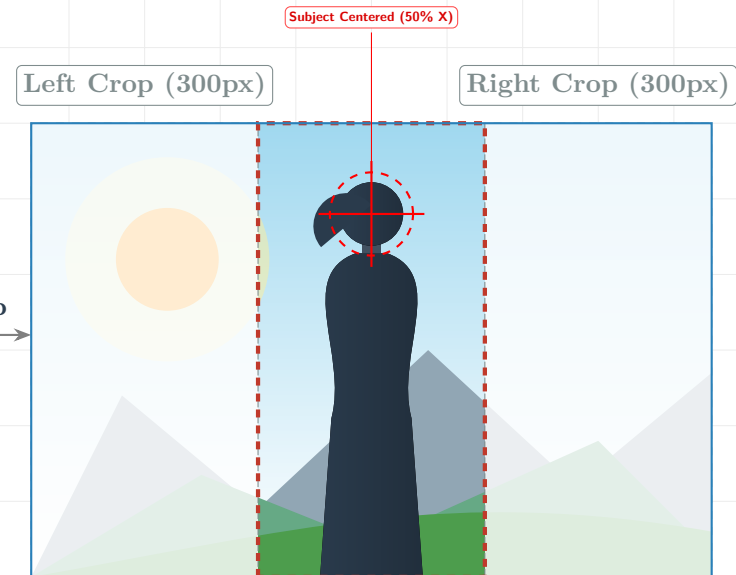
Placed into



Scale & Crop

3. Algorithmic Scale & Crop

Resulting Dimensions: 900px × 600px (1.5x)

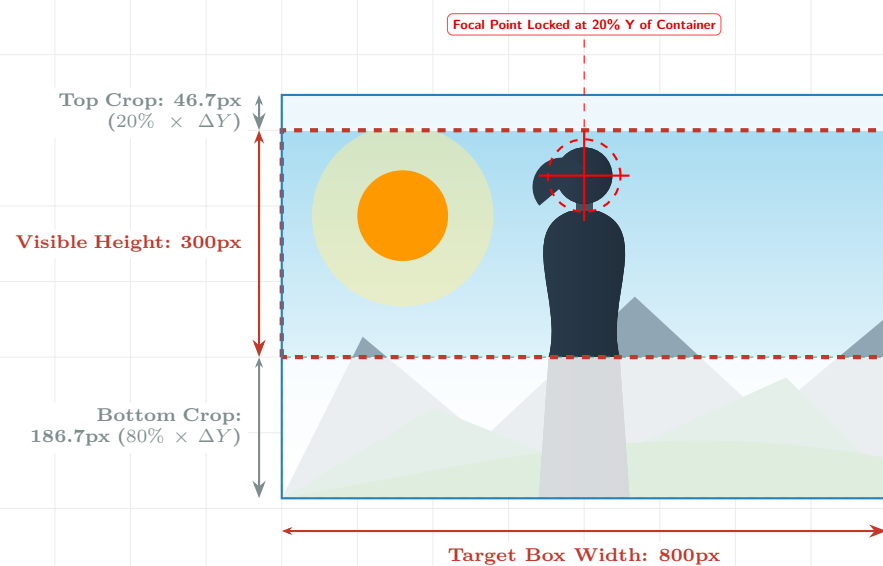


Visible Area (Center 300px)

**At 1:2 ratio, Y-overflow is 0px (Scale_H dictates). The 20% Y-position is dormant here.*

4. The 50% 20% Focus Point Lock (Responsive Horizontal Container)

Container: 800px × 300px (8:3) ⇒ Width Dictates Scale (1.333x), Activating the Vertical 20% Y-Crop



⊕ Focal Point Alignment Math (50% 20%)

When horizontal aspect ratio causes vertical overflow ($\Delta Y > 0$), the Y-offset formula actively shifts the image up to anchor the subject's face:

- **Scale Factor:** $\max\left(\frac{800}{600}, \frac{300}{400}\right) = \max(1.333, 0.75) = 1.333x$
- **Scaled Size:** $800px \times 533.3px \Rightarrow \Delta Y = 533.3 - 300 = 233.3px$
- **Top Crop** = $\Delta Y \times 20\% = 233.3px \times 0.20 = 46.7px$
- **Bottom Crop** = $\Delta Y \times 80\% = 233.3px \times 0.80 = 186.7px$

Result: Face remains at exactly 20% from the container top!

✂ The Core Directive of cover

The algorithm must scale the image so its dimensions are $\geq$ the container's dimensions, strictly preserving the aspect ratio. The dimension requiring the *larger* multiplier dictates the global scale factor:

$$\text{Scale}_W = \frac{300}{600} = 0.5 \quad \text{Scale}_H = \frac{600}{400} = 1.5$$

$$\text{Final Scale Factor} = \max(0.5, 1.5) = 1.5x$$

✂ The Default 50% Crop

By default, object-position is 50% 50%. This mathematically centers the image, dividing the overflow equally:

- **Scaled Width** = $600px \times 1.5 = 900px$
- **Total Overflow** = $900px - 300px = 600px$
- **Left / Right Crop** = $600px \div 2 = 300px$